محمدپارسا شاه‌محمدی 403110506

تمرین مهندسی معکوس

ابتدا با استفاده از ابزار gdb بازی را اجرا می‌کنیم.

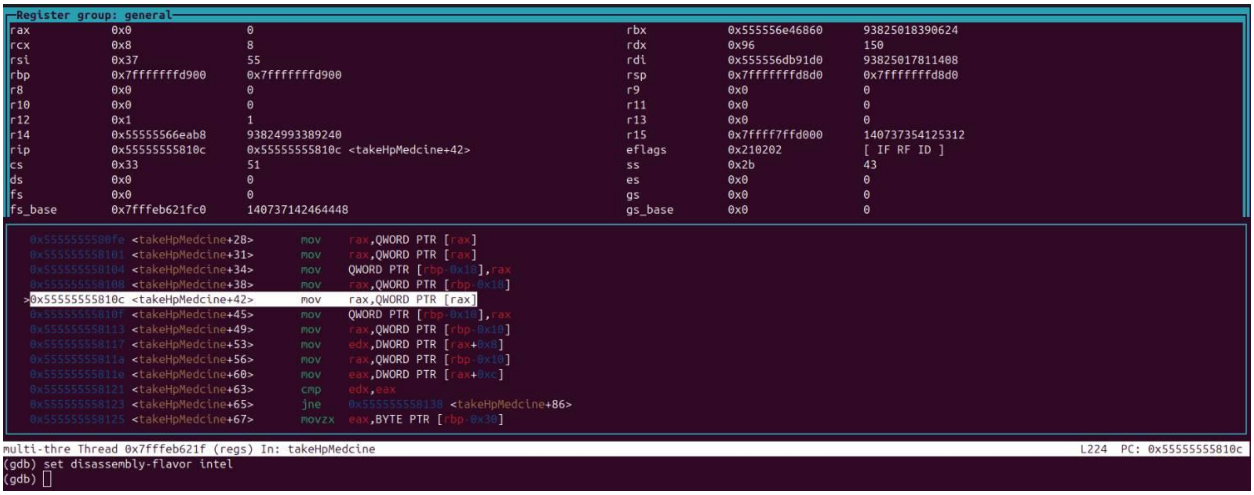


طبق تصویر بالا مشاهده می‌شود هنگامی که آیتم فلاسک را برداشتیم ، دچار خطا شدیم. در هر باری که آیتم فلاسک را برداشتیم خطا صورت نمی‌گیرد و بعضی مواقع با موفقیت آیتم فلاسک را برداشتیم. حال خطایی که رخ داد را در ابزار gdb بررسی می‌کنیم:

```
Thread 1 "game" received signal SIGSEGV, Segmentation fault.
0x000055555555810c in takeHpMedcine (snake=0x555556db91d0, delta=55, extra=false) at /dev/null/src/game.c:224
224     in /dev/null/src/game.c
(gdb) bt
#0  0x000055555555810c in takeHpMedcine (snake=0x555556db91d0, delta=55, extra=false) at /dev/null/src/game.c:224
#1  0x000055555555a764 in makeSnakeCross (snake=0x555556db91d0) at /dev/null/src/game.c:690
#2  0x000055555555b328 in makeCross () at /dev/null/src/game.c:832
#3  0x000055555555c5d9 in gameLoop () at /dev/null/src/game.c:1114
#4  0x00005555555579b2 in startGame (localPlayers=1, remotePlayers=0, localFirst=true) at /dev/null/src/game.c:110
#5  0x00005555555644e0 in launchLocalGame (localPlayerNum=1) at /dev/null/src/ui.c:110
#6  0x00005555555650e3 in mainUi () at /dev/null/src/ui.c:322
#7  0x0000555555565178 in mainUi () at /dev/null/src/ui.c:341
#8  0x0000555555565178 in mainUi () at /dev/null/src/ui.c:341
#9  0x000055555555ce6a in main (argc=1, args=0x7fffffffdcd8) at /dev/null/src/main.c:24
(gdb)
```

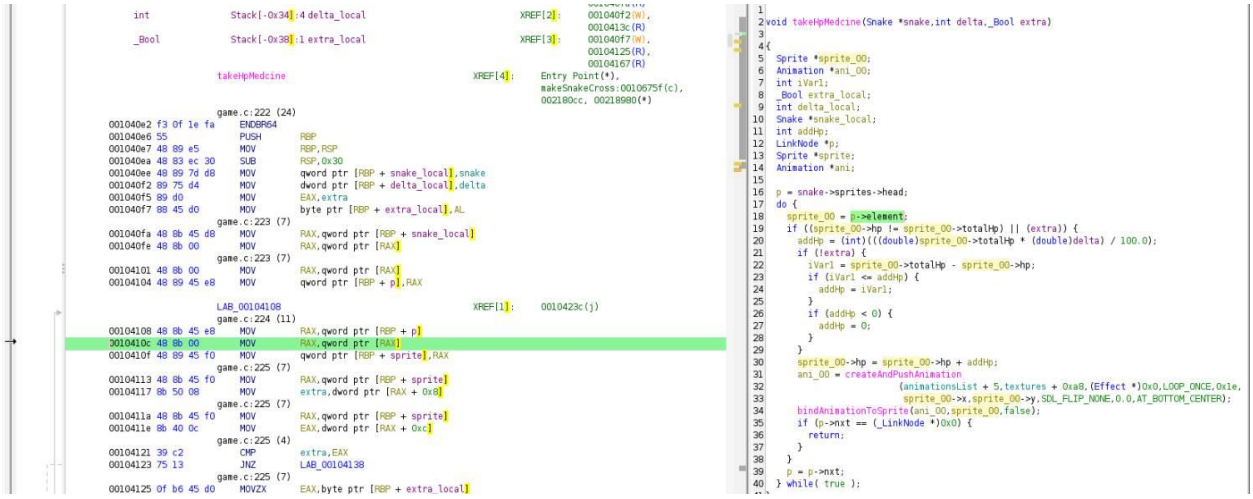
طبق تصویر سیگنال SIGSEGV دریافت شده است که این سیگنال هنگامی که برنامه به یک آدرس حافظه‌ی نامعتبر دسترسی پیدا می‌کند ، فرستاده می‌شود. با استفاده از دستور bt(back trace) در gdb ابزار می‌توان فهمید که خطا در تابع takeHpMedcine صورت گرفته است و این تابع در تابع makeSnakeCross صدا زده شده است.

حال با دستور lay asm می‌توان وضعیت ثبات‌ها و خطی از کد که دچار خطا شده است را دید :



طبق تصویر دستور MOV RAX,QWORD PTR[RAX] در تابع takeHpMedcine باعث بروز خطا شده است. در این دستور به مقداری که داخل خانه‌ای از حافظه که ثبات RAX به آن اشاره دارد در ثبات RAX ریخته می‌شود. طبق جدول مقادیر ثبات‌ها ، مقدار ثبات RAX برابر 0 بوده است و چون دسترسی به خانه 0 حافظه نامعتبر می‌باشد ، برای همین خطا در اجرای این دستور رخ داده است.

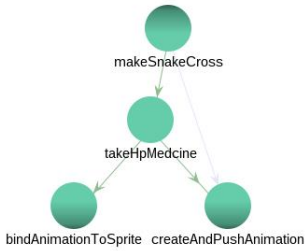
حال با استفاده از ابزار ghidra ، ساختارتابع takeHpmedcine را بررسی می‌کنیم.



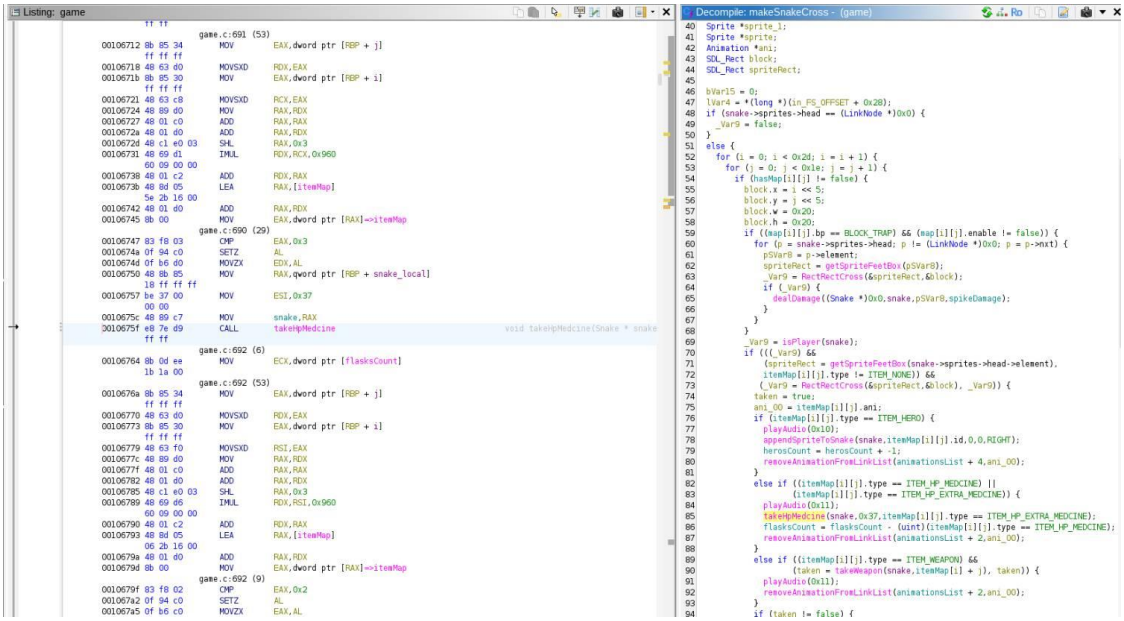
در زبان C معادل نمایشی، دستوری که باعث خطا شده است معادل دستور p->element می‌باشد. طبق تصویر می‌توان فهمید که ابتدا p = snake->sprites->head تعریف شده است (پوینتر به استراکت head می‌باشد). همچنین طبق دستورات در ابتدا ، می‌توان گفت متغییر p انگار همان ثبات RAX می‌باشد و چون فهمیدیم در ثبات RAX مقدار 0 بوده است پس هنگامی که خطا رخ داده است ، p پوینتر NULL بوده است.

می‌توان دید که ساختار تابع takeHpMedcine به گونه‌ای هست که در انتها حلقه p = p->nxt می‌شود و همچنین در بدنه if در خط 35 چک می‌شود که اگر p->nxt برابر 0 بود از تابع خارج می‌شویم.

حال با استفاده از Function Call Graph در بخش Window می‌توان دید که این تابع فقط در تابع makeSnakeCross صدا زده می‌شود و فقط نیاز هست که ساختار این تابع را بررسی کنیم.



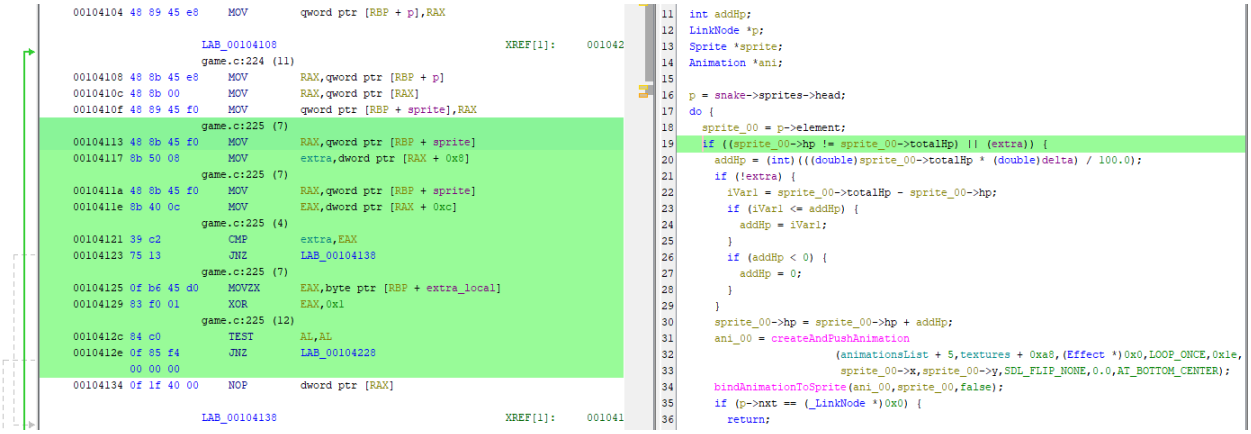
حال در ابزار ghidra ساختار تابع makeSnakeCross را بررسی می‌کنیم.



داخل if در خط 48 چک می‌شود که snake->sprites->head برابر 0 هست و اگر نباشد به داخل else می‌رویم که در این بخش در خط 85 تابع takeHpMedcine صدا زده می‌شود. بقیه دستورات هم snake را تغییر نمی‌دهند ، پس درمورد snake که به تابع takeHpMedcine ورودی داده می‌شود اطمینان داریم که snake->sprites->head برابر 0 نمی باشد.

طبق تصویری که از ساختار تابع takeHpMedcine دیدیم ، می توان گفت که هنگامی که p تعریف شده است چون می‌دانیم snake->sprites->head در ابتدا 0 نمی‌باشد ، پس پوینتر NULL نخواهد بود. پس در دفعه اولی که حلقه اجرا می‌شود ، دچار خطایی شناسایی شده نخواهیم شد. پس خطا به خاطر دستور p = p->nxt در انتهای حلقه می‌باشد چراکه همواره این که p->nxt ، 0 نباشد چک نمی‌شود و فقط زمانی چک می‌شود شرط داخل if خط 19 برقرار باشد.

راهکار : می‌توان برای حل مشکل دستوراتی که ناصفر بودن p->nxt را چک می‌کنند را از داخل بدنه if خط 19 خارج کنیم و همواره پیش از این که دستورp = p->nxt انجام شود ، ناصفر بودن p->nxt چک شود.



```
00104104 48 89 45 e8     MOV        qword ptr [RBP + p],RAX

                     LAB_00104108                                    XREF[1]:     001042
                     game.c:224 (11)
00104108 48 8b 45 e8     MOV        RAX,qword ptr [RBP + p]
0010410c 48 8b 00        MOV        RAX,qword ptr [RAX]
0010410f 48 89 45 f0     MOV        qword ptr [RBP + sprite],RAX
                     game.c:225 (7)
00104113 48 8b 45 f0     MOV        RAX,qword ptr [RBP + sprite]
00104117 8b 50 08        MOV        extra,dword ptr [RAX + 0x8]
                     game.c:225 (7)
0010411a 48 8b 45 f0     MOV        RAX,qword ptr [RBP + sprite]
0010411e 8b 40 0c        MOV        EAX,dword ptr [RAX + 0xc]
                     game.c:225 (4)
00104121 39 c2           CMP        extra,EAX
00104123 75 13           JNZ        LAB_00104138
                     game.c:225 (7)
00104125 0f b6 45 d0     MOVZX      EAX,byte ptr [RBP + extra_local]
00104129 83 f0 01        XOR        EAX,0x1
                     game.c:225 (12)
0010412c 84 c0           TEST       AL,AL
0010412e 0f 85 f4        JNZ        LAB_00104228
         00 00 00
00104134 0f 1f 40 00     NOP        dword ptr [RAX]

                     LAB_00104138                                    XREF[1]:     001041
```

```
int addHp;
LinkNode *p;
Sprite *sprite;
Animation *ani;

p = snake->sprites->head;
do {
  sprite_00 = p->element;
  if ((sprite_00->hp != sprite_00->totalHp) || (extra)) {
    addHp = (int)(((double)sprite_00->totalHp * (double)delta) / 100.0);
    if (!extra) {
      iVar1 = sprite_00->totalHp - sprite_00->hp;
      if (iVar1 <= addHp) {
        addHp = iVar1;
      }
      if (addHp < 0) {
        addHp = 0;
      }
    }
    sprite_00->hp = sprite_00->hp + addHp;
    ani_00 = createAndPushAnimation
                       (animationsList + 5,textures + 0xa8,(Effect *)0x0,LOOP_ONCE,0x1e,
                        sprite_00->x,sprite_00->y,SDL_FLIP_NONE,0.0,AT_BOTTOM_CENTER);
    bindAnimationToSprite(ani_00,sprite_00,false);
    if (p->nxt == (_LinkNode *)0x0) {
      return;
```

طبق تصویر در بخش سبز شده ، شرط داخل if بررسی می‌شود و اگر برقرار نبود(بر اساس وضعیت پرچم Z بعد از دستور TEST AL,AL) با دستور JNZ LAB_00104228 به خط 00104228 پرش می‌کنیم. (این خط در تصویر پایین معلوم هست) بعد از پرش به این خط دستور p = p->nxt انجام می‌شود (دستورات NOP(NO-Operation) کاری انجام نمی‌دهند و فقط دستورات MOV مهم هستند) و بعد با دستور JMP LAB_00104108 به خط 00104108 که عملا اول حلقه می‌باشد برمی‌گردیم.



```
00104211 48 89 c7        MOV        snake,RAX
00104214 e8 aa 88        CALL       bindAnimationToSprite                            void
         00 00
                     game.c:234 (8)
00104219 48 8b 45 e8     MOV        RAX,qword ptr [RBP + p]
0010421d 48 8b 40 10     MOV        RAX,qword ptr [RAX + 0x10]
                     game.c:234 (7)
00104221 48 85 c0        TEST       RAX,RAX
00104224 74 22           JZ         LAB_00104248
00104226 eb 08           JMP        LAB_00104230

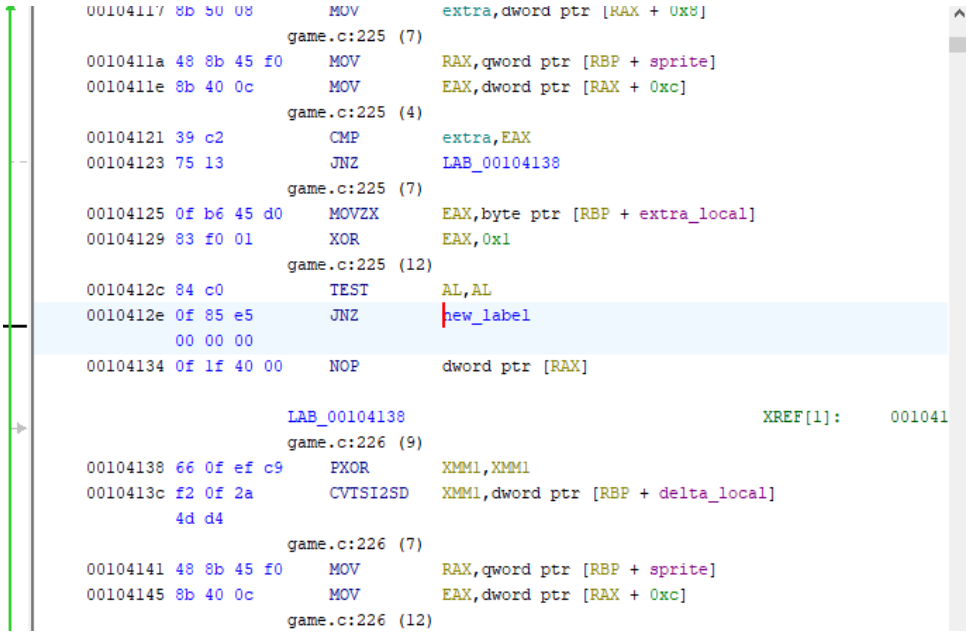
                     LAB_00104228                                    XREF[1]:     0010412e
                     game.c:225 (8)
00104228 90              NOP
00104229 0f 1f 80        NOP        dword ptr [RAX]
         00 00 00 00

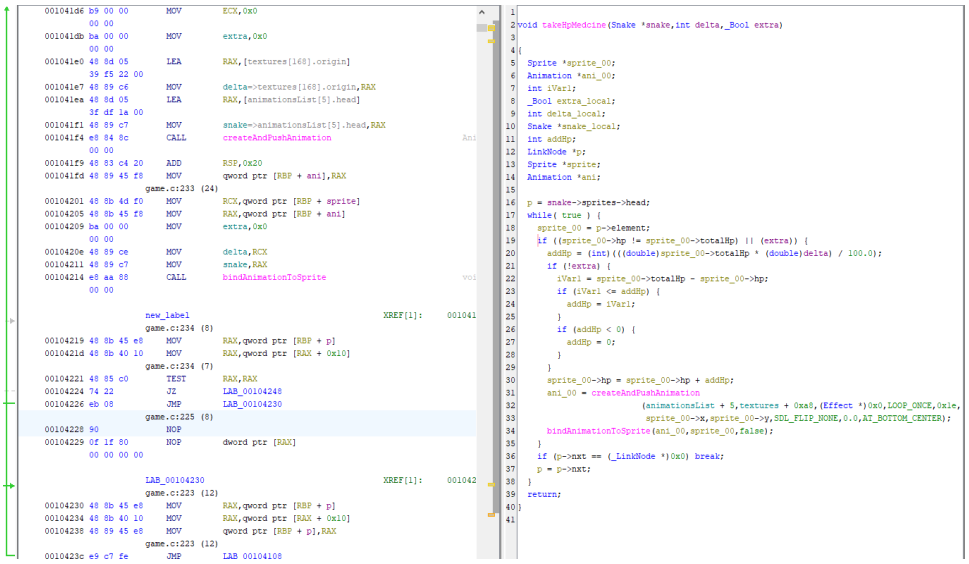
                     LAB_00104230                                    XREF[1]:     00104226
                     game.c:223 (12)
00104230 48 8b 45 e8     MOV        RAX,qword ptr [RBP + p]
00104234 48 8b 40 10     MOV        RAX,qword ptr [RAX + 0x10]
00104238 48 89 45 e8     MOV        qword ptr [RBP + p],RAX
                     game.c:223 (12)
0010423c e9 c7 fe        JMP        LAB_00104108
         ff ff
```

```
do {
  sprite_00 = p->element;
  if ((sprite_00->hp != sprite_00->totalHp) || (extra)) {
    addHp = (int)(((double)sprite_00->totalHp * (double)delta) / 100.0);
    if (!extra) {
      iVar1 = sprite_00->totalHp - sprite_00->hp;
      if (iVar1 <= addHp) {
        addHp = iVar1;
      }
      if (addHp < 0) {
        addHp = 0;
      }
    }
    sprite_00->hp = sprite_00->hp + addHp;
    ani_00 = createAndPushAnimation
                       (animationsList + 5,textures + 0xa8,(Effect *)0x0,LOOP_ONCE,
                        sprite_00->x,sprite_00->y,SDL_FLIP_NONE,0.0,AT_BOTTOM_CENTE
    bindAnimationToSprite(ani_00,sprite_00,false);
    if (p->nxt == (_LinkNode *)0x0) {
      return;
    }
  }
  p = p->nxt;
} while( true );
```

طبق تصویر در بخش سبز شده ، شرط ناصفر بودن چک می‌شود. اگر شرط برقرار بود (بر اساس وضعیت پرچم Z بعد از دستور TEST RAX,RAX) با دستور JZ LAB_00104248 از تابع خارج می‌شویم. ولی اگر شرط برقرار نبود با دستور JMP LAB_00104230 به خط 00104230 پرش می‌کنیم و بعد در ادامه دستور p = p->nxt اجرا می‌شود و در انتها به اول حلقه پرش می‌کنیم.

حال برای درست کردن یک label به نام new_label در خط 00104219 تعریف می‌کنیم (در خطی که شروع به چک کردن شرط ناصفر بودن p->nxt می‌کنیم) و به جای دستور JNZ LAB_00104228 در خط 0010412e که برای if در ابتدا حلقه بود(در تصویر بالایی) ، دستورJNZ new_label قرار می‌دهیم. کد اصلاح شده :





طبق شکل جدید همواره در حلقه شرط ناصفر بودن p->nxt قبل از دستور p = p->nxt چک می‌شود ؛ فارغ از این که شرط داخل if اول حلقه برقرار هست یا نه. درنتیجه همواره اطمینان داریم که بعد از انجام دستور p = p->nxt و بازگشت به ابتدا حلقه ، p پوینتر NULL نیست و در دستور MOV RAX,QWORD PTR[RAX] که قبلا دچار خطا می‌شدیم ، دیگر دچار خطا نخواهیم شد.

نکته : به خاطر این که تابع takeHpMedcine فقط در تابع makeSnakeCross صدا زده می‌شد و در آن تابع هم می‌دانستیم snake->sprites->head ناصفر است توانستیم این کار را انجام دهیم. در غیر این صورت باید حلقه را تبدیل به for می‌کردیم که در ابتدای هر حلقه ناصفر بودن p->nxt چک شود.